

```

import React, {
  forwardRef,
  useState,
  useEffect,
  useRef,
  useCallback,
  useMemo,
} from "react";
import { LucideIcon } from "lucide-react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";
import { HintBox } from "../feedback/hint-box";
import { PremiumCheckbox } from "../checkbox";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

export interface ListBoxOption extends Record<string, unknown> {
  id?: string | number;
  name?: string;
}

export interface ListBoxRightActionConfig {
  icon: LucideIcon;
  onClick: (
    e: React.MouseEvent<HTMLButtonElement>,
    optionItem: ListBoxOption,
  ) => void;
  tooltipText?: string;
  className?: string;
}

export interface ListBoxProps extends Omit<
  React.HTMLAttributes<HTMLDivElement>,
  "onChange" | "value"
> {
  label?: string;
  error?: string;
  hint?: string;
  options: ListBoxOption[];
  value?: unknown | unknown[];
  onChange?: (selectedValue: unknown, fullRecordRows: unknown) => void;
  multiple?: boolean;
  displayKey?: string;
  valueKey?: string;
  rightActionButtons?: ListBoxRightActionConfig[];
  disabled?: boolean;
}

```

```

export const ListBox = forwardRef<HTMLDivElement, ListBoxProps>(
  (
    {
      label,
      error,
      hint,
      options = [],
      value,
      onChange,
      multiple = false,
      displayKey = "name",
      valueKey = "id",
      rightActionButtons = [],
      disabled = false,
      className,
      ...props
    },
    ref,
  ) => {
    const containerRef = useRef<HTMLDivElement>(null);
    const [focusedIndex, setFocusedIndex] = useState<number>(-1);
    const [selectedValues, setSelectedValues] = useState<unknown[]>([]);

    const normalizedValue = useMemo(() => {
      if (value === undefined || value === null) return [];
      return Array.isArray(value) ? value : [value];
    }, [value]);

    useEffect(() => {
      setSelectedValues(normalizedValue);
    }, [normalizedValue]);

    const getOptionValue = useCallback(
      (option: ListBoxOption): unknown => {
        return option[valueKey] ?? option[displayKey] ?? option;
      },
      [valueKey, displayKey],
    );

    const getOptionLabel = useCallback(
      (option: ListBoxOption): string => {
        return String(option[displayKey] ?? option[valueKey] ?? "");
      },
      [displayKey, valueKey],
    );

    const isOptionSelected = useCallback(
      (option: ListBoxOption): boolean => {
        const optValue = getOptionValue(option);
        return selectedValues.some((v) => v === optValue);
      }
    );
  }
);

```

```

    },
    [selectedValues, getOptionValue],
  );

const handleSelectionChange = useCallback(
  (option: ListBoxOption) => {
    const optValue = getOptionValue(option);
    const isSelected = isOptionSelected(option);

    let newSelectedValues: unknown[];
    if (multiple) {
      newSelectedValues = isSelected
        ? selectedValues.filter((v) => v !== optValue)
        : [...selectedValues, optValue];
    } else {
      newSelectedValues = isSelected ? [] : [optValue];
    }

    setSelectedValues(newSelectedValues);
    onChange?.(multiple ? newSelectedValues : newSelectedValues[0], option);
  },
  [multiple, selectedValues, getOptionValue, isOptionSelected, onChange],
);

const handleKeyDown = useCallback(
  (e: React.KeyboardEvent<HTMLDivElement>, index: number) => {
    switch (e.key) {
      case "ArrowDown":
        e.preventDefault();
        setFocusedIndex((prev) => Math.min(prev + 1, options.length - 1));
        break;
      case "ArrowUp":
        e.preventDefault();
        setFocusedIndex((prev) => Math.max(prev - 1, 0));
        break;
      case "Enter":
      case " ":
        e.preventDefault();
        if (index >= 0 && index < options.length) {
          handleSelectionChange(options[index]);
        }
        break;
      case "Escape":
        setFocusedIndex(-1);
        break;
    }
  },
  [options, handleSelectionChange],
);

```

```

const handleRowClick = useCallback(
  (
    e: React.MouseEvent<HTMLButtonElement>,
    option: ListBoxOption,
    index: number,
  ) => {
    if (rightActionButtons.length > 0) {
      const target = e.target as HTMLElement;
      const isActionButton = target.closest('[data-action-button="true"]');
      if (!isActionButton) {
        handleSelectionChange(option);
      }
    } else {
      handleSelectionChange(option);
    }
    setFocusedIndex(index);
  },
  [rightActionButtons, handleSelectionChange],
);

const handleActionClick = useCallback(
  (
    e: React.MouseEvent<HTMLButtonElement>,
    action: ListBoxRightActionConfig,
    option: ListBoxOption,
  ) => {
    e.stopPropagation();
    action.onClick(e, option);
  },
  [],
);

const handleFocus = useCallback((index: number) => {
  setFocusedIndex(index);
}, []);

const handleBlur = useCallback(() => {
  setFocusedIndex(-1);
}, []);

const limitedRightActions = useMemo(
  () => rightActionButtons.slice(0, 2),
  [rightActionButtons],
);

const hasError = Boolean(error);

return (
  <div ref={ref} className={cn("w-full", className)} {...props}>
    {label} && (

```

```

<label
  className={cn(
    "block text-sm font-medium text-text-main mb-1.5",
    disabled && "opacity-50",
  )}
>
  {label}
</label>
)}
{hint && (
  <HintBox content={hint} className="mb-1.5">
    {hint}
  </HintBox>
)}
<div
  ref={containerRef}
  className={cn(
    "bg-card/90 backdrop-blur-[var(--backdrop-blur)]",
    "border-[color-mix(in_oklch,var(--color-border)_40%,transparent)]",
    "rounded-surface",
    "overflow-hidden",
    "focus-within:ring-2 focus-within:ring-amber-500/20",
    "focus-within:border-amber-500 focus-within:bg-amber-500/5",
    "disabled:opacity-50 disabled:pointer-events-none",
    "disabled:cursor-not-allowed",
    "transition-all duration-200 ease-out",
    hasError &&
    "border-destructive/50 focus-within:ring-destructive/50",
    "focus-within:border-destructive/50",
    disabled && "bg-muted/50",
  )}
  role="listbox"
  aria-multiselectable={multiple}
  tabIndex={disabled ? -1 : 0}
  onKeyDown={(e) => handleKeyDown(e, focusedIndex)}
  onFocus={() => handleFocus(0)}
  onBlur={handleBlur}
>
  {options.length === 0 ? (
    <div className="px-4 py-8 text-center text-muted-foreground text-sm">
      No options available
    </div>
  ) : (
    <div role="listbox" aria-label="Options">
      {options.map((option, index) => {
        const isSelected = isOptionSelected(option);
        const isFocused = index === focusedIndex;
        const optValue = getOptionValue(option);
        const optLabel = getOptionLabel(option);

```

```

return (
  <button
    key={String(optValue)}
    type="button"
    role="option"
    aria-selected={isSelected}
    aria-highlighted={isFocused}
    tabIndex={-1}
    onClick={(e) => handleRowClick(e, option, index)}
    onFocus={() => handleFocus(index)}
    onBlur={handleBlur}
    className={cn(
      "w-full px-4 py-3 text-left text-sm",
      "flex items-center gap-3",
      "transition-colors duration-100",
      "focus:outline-none focus:bg-brand-primary/5",
      hover:bg-brand-primary/5",
      isFocused && "bg-brand-primary/5 text-text-main",
      isSelected &&
        "bg-brand-primary/10 text-brand-primary font-semibold",
      border-1-2 border-brand-primary",
      disabled &&
        "opacity-50 pointer-events-none cursor-not-allowed",
    )}
  >
    {multiple && (
      <PremiumCheckbox
        checked={isSelected}
        onChange={() => handleSelectionChange(option)}
        disabled={disabled}
        className="shrink-0"
        aria-hidden="true"
      />
    )}
    <span
      className={cn(
        "truncate flex-1 min-w-0",
        isSelected && "font-semibold",
      )}
    >
      {optLabel}
    </span>
    {limitedRightActions.length > 0 && (
      <div
        className="flex items-center gap-1.5 shrink-0"
        data-action-buttons="true"
      >
        {limitedRightActions.map((action, actionIndex) => (
          <button
            key={actionIndex}

```

```

        type="button"
        data-action-button="true"
        className={cn(
            "relative flex items-center justify-center",
            "w-8 h-8 rounded-md",
            "text-muted-foreground/60 hover:text-foreground",
            "bg-transparent hover:bg-accent",
            "transition-transform duration-100",
            "active:scale-95",
            "focus:outline-none focus-visible:ring-2
focus-visible:ring-amber-500/20 focus-visible:border-amber-500",
            "disabled:opacity-50 disabled:pointer-events-none",
            action.className,
        )}
        onClick={(e) =>
            handleActionClick(e, action, option)
        }
        aria-label={action.tooltipText || "Action"}
        disabled={disabled}
    >
        <action.icon
            className="w-4 h-4"
            aria-hidden="true"
        />
    </button>
    )}
</div>
)}
</button>
);
}}
</div>
)}
</div>
{error && (
    <p
        className={cn(
            "mt-1.5 text-sm",
            "text-destructive/90",
            "font-medium",
        )}
        role="alert"
    >
        {error}
    </p>
    )}
</div>
);
},
);

```

```
ListBox.displayName = "ListBox";
```